

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS — 1ère année — Département de français

Durée : 1h30

Année universitaire : 2025-2026

Objectifs pédagogiques

À l'issue de ce TP, l'étudiant est capable de :

- Maîtriser la boucle `for` avec `range()`
- Maîtriser la boucle `while` avec condition d'arrêt
- Utiliser `break` et `continue` pour contrôler l'exécution
- Implémenter des algorithmes itératifs (somme, suite, jeu)
- Valider des saisies répétées avec compteur de tentatives

Plan du TP

Phase	Contenu	Durée
Accueil	Rappel sur les boucles	10 min
Exercice 1	Table de multiplication	15 min
Exercice 2	Somme des N premiers entiers	15 min
Exercice 3	Suite de Fibonacci	20 min
Exercice 4	Devinette numérique	20 min
Exercice 5	Validation répétée (mot de passe × 3)	10 min

Prérequis

- TP 01 et TP 02 bouclés (variables, conditions, `try/except`)
- Notion de `range()` et de boucle vues en cours

Exercice 1 — Table de multiplication (15 min)

Énoncé : Créer un script `tables.py` qui affiche la table de multiplication d'un nombre saisi, de 1 à 10, en utilisant une boucle `for`.

Exemple attendu :

```
Nombre : 7
7 × 1 = 7
7 × 2 = 14
...
7 × 10 = 70
```

Corrigé :

```
try:
    n = int(input("Nombre : "))
    for i in range(1, 11):
        print(f"{n} × {i} = {n * i}")
    except ValueError:
        print("Veuillez entrer un nombre entier.")
```

Diagramme d'une boucle for :

```
flowchart TD
    A[Début] --> B[Définir n]
    B --> C["Initialiser i = 1"]
    C --> D{"i ≤ 10 ?"}
    D -->|Oui| E[Afficher n × i]
    E --> F["Incrémenter i (i + 1)"]
    F --> D
    D -->|Non| G[Fin]
```

Exercice 2 — Somme des N premiers entiers (15 min)

Énoncé : Créer un script `somme_n.py` qui :

1. Demande un entier N à l'utilisateur
2. Calcule la somme des entiers de 1 à N avec une boucle for
3. Calcule la somme des entiers pairs de 1 à N
4. Vérifie le résultat avec la formule mathématique $S = N(N+1)/2$

Exemple attendu :

```
N = 10
Somme 1 à 10 : 55
Somme des pairs : 30
Formule : 55 – Vérifié ✓
```

Corrigé :

```
try:
    N = int(input("N = "))
    somme = 0
    somme_pairs = 0
    for i in range(1, N + 1):
        somme += i
        if i % 2 == 0:
            somme_pairs += i
    formule = N * (N + 1) // 2
    print(f"Somme 1 à {N} : {somme}")
    print(f"Somme des pairs : {somme_pairs}")
    print(f"Formule : {formule}", end=" – ")
    print("Vérifié ✓" if somme == formule else "Erreur x")
except ValueError:
    print("Veuillez entrer un entier.")
```

Exercice 3 — Suite de Fibonacci (20 min)

Énoncé : Créer un script `fibonacci.py` qui affiche les N premiers termes de la suite de Fibonacci (N saisi par l'utilisateur). $F(0) = 0$, $F(1) = 1$, $F(n) = F(n-1) + F(n-2)$.

Exemple attendu (n = 10) :

Nombre de termes : 10
0, 1, 1, 2, 3, 5, 8, 13, 21, 34

Corrigé :

```
try:
    n = int(input("Nombre de termes : "))
    a, b = 0, 1
    for _ in range(n):
        print(a, end=" ")
        a, b = b, a + b
    print()
except ValueError:
    print("Veuillez entrer un entier.")
```

Exercice 4 — Devinette numérique (20 min)

Énoncé : Créer un script devine.py qui :

1. Choisit un nombre aléatoire entre 1 et 100
2. Demande à l'utilisateur de deviner
3. Indique si le nombre est trop petit ou trop grand
4. Limite à 7 tentatives maximum
5. Affiche le nombre secret si perdu

Exemple attendu :

```
Devinez le nombre entre 1 et 100 (7 tentatives max).
Votre proposition : 50
Trop grand.
Votre proposition : 25
Trop petit.
...
```

Diagramme de flux de la devinette :

```
flowchart TD
    A[Début] --> B["Générer nombre secret\n(1 à 100)"]
    B --> C["Initialiser tentatives = 0"]
    C --> D{"tentatives < 7 ?"}
    D -->|Oui| E[Saisir proposition]
    E --> F[Inc. tentatives]
    F --> G{"Proposition = secret ?"}
    G -->|Oui| H[Afficher Bravo !]
    G -->|Non| I{"Proposition < secret ?"}
    I -->|Oui| J[Afficher Trop petit]
    I -->|Non| K[Afficher Trop grand]
    J --> D
    K --> D
    D -->|Non| L[Afficher Perdu + secret]
    H --> M[Fin]
    L --> M
```

Corrigé :

```
import random

secret = random.randint(1, 100)
tentatives = 0
max_tentatives = 7
```

```

print("Devinez le nombre entre 1 et 100 (7 tentatives max).")

while tentatives < max_tentatives:
    try:
        essai = int(input("Votre proposition : "))
        tentatives += 1
        if essai == secret:
            print(f"Bravo ! Trouvé en {tentatives} tentative(s).")
            break
        elif essai < secret:
            print("Trop petit.")
        else:
            print("Trop grand.")
        except ValueError:
            print("Entrez un nombre valide.")

    if tentatives == max_tentatives:
        print(f"Perdu ! Le nombre était {secret}.")

```

Exercice 5 — Validation répétée (mot de passe × 3) (10 min)

Énoncé : Créer un script `mdp_repet.py` qui :

1. Définit un mot de passe secret fixe (ex: "ENS2026")
2. Demande à l'utilisateur de saisir le mot de passe
3. Accorde 3 tentatives maximum
4. Affiche « Accès autorisé » ou « Compte bloqué »

Exemple attendu :

```

Mot de passe : python
Mot de passe incorrect (2 tentative(s) restante(s))
Mot de passe : 1234
Mot de passe incorrect (1 tentative(s) restante(s))
Mot de passe : ENS2026
Accès autorisé ✓

```

Corrigé :

```

mot_de_passe = "ENS2026"
tentatives = 3

while tentatives > 0:
    saisie = input("Mot de passe : ")
    if saisie == mot_de_passe:
        print("Accès autorisé ✓")
        break
    else:
        tentatives -= 1
        if tentatives > 0:
            print(f"Mot de passe incorrect ({tentatives} tentative(s) restante(s))")
        else:
            print("Compte bloqué – trop de tentatives.")

```

Diagramme d'une boucle while :

```

flowchart TD
    A[Début] --> B["Initialiser tentatives = 3"]
    B --> C{"tentatives > 0 ?"}
    C -->|Oui| D[Saisir mot de passe]
    D --> E{"Correct ?"}

```

```

E -->|Oui| F[Accès autorisé]
F --> G[Fin]
E -->|Non| H[Décrémenter tentatives]
H --> I{tentatives > 0 ?}
I -->|Oui| J[Afficher tentative restante]
J --> C
I -->|Non| K[Compte bloqué]
K --> G

```

QCM d'auto-évaluation

1. Combien de fois la boucle `for i in range(5)` : s'exécute-t-elle ?

- a) 4 fois
- b) 5 fois ✓
- c) 6 fois

2. Que fait l'instruction `break` dans une boucle `while` ?

- a) Elle saute une itération
- b) Elle arrête la boucle immédiatement ✓
- c) Elle redémarre la boucle

3. Quelle est la somme des entiers de 1 à 100 avec la formule ?

- a) 4950
- b) 5050 ✓
- c) 5150

4. Dans la suite de Fibonacci, que vaut $F(5)$ si $F(0)=0$ et $F(1)=1$?

- a) 3
- b) 5 ✓
- c) 8

5. Quelle condition arrête une boucle `while True` : ?

- a) `continue`
- b) `break` ✓
- c) `pass`

Grille d'évaluation TP 03

Critère	Poids	Non
Boucle <code>for</code> et <code>range()</code>	20 %	Ne m
Boucle <code>while</code> et condition	20 %	Ne m
<code>break</code> et contrôle de flux	20 %	N'util
Algorithmes itératifs	20 %	Aucur
Gestion des tentatives	20 %	Aucur

Déroulement séance — 1h30 (90 min)

Phase	Durée	Total cumulé
Accueil et rappel boucles	10 min	10 min
Exercice 1 — Table de multiplication	15 min	25 min
Exercice 2 — Somme des N premiers	15 min	40 min
Exercice 3 — Suite de Fibonacci	20 min	60 min
Exercice 4 — Devinette numérique	20 min	80 min
Exercice 5 — Validation répétée	10 min	90 min

Note pédagogique ENS

Ce TP sur les boucles et itérations est crucial pour développer la pensée algorithmique des futurs enseignants. Chaque exercice illustre un usage distinct des structures répétitives. La table de multiplication (ex. 1) est l'exemple le plus simple pour introduire for et peut être présentée dès la 4e. La somme des N premiers entiers (ex. 2) permet un aller-retour fructueux entre algorithmique et mathématiques (formule close vs calcul itératif). La suite de Fibonacci (ex. 3) est un classique interdisciplinaire (mathématiques, biologie, architecture). La devinette numérique (ex. 4) introduit le jeu comme vecteur d'apprentissage — les élèves apprennent sans s'en rendre compte. La validation répétée (ex. 5) sensibilise à la sécurité (brute-force, verrouillage de compte). L'enseignant peut demander aux étudiants de concevoir un exercice similaire adapté à un niveau collège, développant ainsi leur capacité de transposition didactique.

Prolongement possible : Ajouter un mode difficile (10 tentatives max, intervalle 1-500), enregistrer le score dans un fichier, ou créer un générateur de tables de multiplication aléatoires pour élèves de CE2/CM1.